Batch Normalization in Deep Learning | Batch Learning in Keras

➢ Introduction:

As neural networks go deeper, training becomes difficult due to **unstable activations** and **exploding/vanishing gradients**.

Batch Normalization (BN) was introduced to **make deep networks train faster, more stable, and more accurate**.

➢ What is Batch Normalization?

Definition:

Batch Normalization is a technique that **normalizes the output of each layer** (or sometimes the input) so that its values have a consistent distribution typically mean = 0 and variance = 1.

It's applied **within each mini-batch** during training.

In simple terms:

It ensures that the data going into each layer stays in a stable range not too large, not too small throughout training.

➢ Why Use Batch Normalization?

Training deep networks is hard because:

- Activations can explode or vanish.
- Small changes in weights can cause big shifts in later layers.

Batch Normalization helps by:

1. Keeping activations stable
2. Allowing higher learning rates
3. Reducing dependence on careful weight initialization
4. Acting as a regularizer (reduces overfitting)

➢ The Problem: Internal Covariate Shift

What is it?

As training progresses, the **distribution of activations in hidden layers keeps changing** because weights in earlier layers are being updated.

This constant change makes training slow — because each layer has to keep adapting to new distributions from the previous layer.

Batch Normalization fixes this by re-centering and re-scaling the activations within each batch.

Batch Normalization: The How?

1 Compute batch mean:

$$\mu_B = \frac{1}{m}\sum_i x_i$$

2 Compute batch variance:

$$\sigma_B^2 = \frac{1}{m}\sum_i (x_i - \mu_B)^2$$

3 Normalize:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

4 Scale and shift (using learnable parameters):

$$y_i = \gamma \hat{x}_i + \beta$$

Here:

- γ: scale parameter (learned)
- β: shift parameter (learned)
- ϵ: small number to prevent division by zero

➢ Where is it Applied?

Usually between:

Linear/Conv Layer → BatchNorm → Activation (ReLU/Sigmoid)

➢ Batch Normalization During Testing

During training:

- BN uses the **mean and variance** of the **current batch**.

During testing (inference):

- BN uses **moving averages** (running mean & variance) calculated during training. This ensures consistency in predictions even when batches aren't available.

➢ Advantages of Batch Normalization

1. Reduces internal covariate shift
2. Allows higher learning rates → faster convergence
3. Acts as a regularizer → less overfitting
4. Makes networks more stable
5. Reduces sensitivity to weight initialization
6. Enables deeper architectures

➢ Outro / Key Takeaways

Batch Normalization = Normalize activations layer-wise

It speeds up training and improves model performance

Works like a stabilizer for deep neural networks

Works both as a **normalizer** and a **regularizer**

➢ Quick Revision Summary Table

Step	Formula / Concept	Purpose
Mean	$\mu_B = \frac{1}{m}\sum x_i$	Compute average
Variance	$\sigma_B^2 = \frac{1}{m}\sum(x_i - \mu_B)^2$	Compute spread
Normalize	$\hat{x}_i = (x_i - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$	Zero mean, unit variance
Scale & Shift	$y_i = \gamma\hat{x}_i + \beta$	Learn new mean/scale

➢ Real-world Analogy

Think of Batch Normalization like **air conditioning** in a factory:

It keeps the internal environment stable, so all machines (layers) work efficiently regardless of the outside temperature (input variation).